

SQL Notes for Beginners - Part-1

1. What is SQL?

- SQL (Structured Query Language) is a standard language used to manage and manipulate relational databases.
- SQL allows you to create databases, tables, insert, update, delete data, and retrieve data using queries.
- It is declarative, which means you tell the database what you want, not how to do it.

2. Why Learn SQL?

- Data is everywhere – businesses store large amounts of data in databases.
- SQL is used in almost every domain including web development, data analysis, data science, and more.
- It helps you retrieve insights from data efficiently.
- It is easy to learn and very powerful.

3. Types of SQL Commands

SQL commands are categorized into the following groups:

- DDL (Data Definition Language): Used to define and manage database structures.
- DML (Data Manipulation Language): Used to manage data inside database objects.
- DQL (Data Query Language): Used to retrieve data from database.
- DCL (Data Control Language): Used to control access to data.
- TCL (Transaction Control Language): Used to manage transactions in a database.

4. Basic SQL Syntax

A basic SQL query is written using the following keywords:

`SELECT` → `FROM` → `WHERE` → `GROUP BY` → `HAVING` → `ORDER BY`

Example:

```
SELECT   name, age          # Columns to retrieve
FROM     students           # Table to retrieve data from
WHERE    age > 18           # Condition to filter rows
ORDER BY name ASC;          # Sort the result by name (A to Z)
```

5. SQL Statements

SQL statements can be categorized into the following types:

- DDL (Data Definition Language): Define or modify database objects.
- DML (Data Manipulation Language): Manipulate data in tables.
- DQL (Data Query Language): Retrieve data from the database.
- DCL (Data Control Language): Control access to data.
- TCL (Transaction Control Language): Manage transactions.

6. DDL - Data Definition Language

DDL commands are used to define and manage the structure of database objects like databases, tables, and indexes.

- CREATE - Creates a new table

```
CREATE TABLE students (
    id      INT PRIMARY KEY,
    name    VARCHAR(50) NOT NULL,
    age     INT,
    email   VARCHAR(100) UNIQUE
);
```

- PRIMARY KEY: Uniquely identifies each record.
- NOT NULL: Column cannot have NULL values.
- UNIQUE: All values in the column must be unique.

- ALTER - Modifies an existing table

```
ALTER TABLE students ADD COLUMN phone VARCHAR(15);
```

- DROP - Deletes a table

```
DROP TABLE students;
```

- TRUNCATE - Removes all records from a table

```
TRUNCATE TABLE students;
```

7. DML - Data Manipulation Language

DML commands are used to manage data within tables.

- INSERT INTO - Adds new records into a table.

```
INSERT INTO students (id, name, age)
VALUES (1, 'Alice', 22);
```
Insert one record

- UPDATE - Modifies existing records.

```
UPDATE students
SET age = 23
WHERE name = 'Alice';
```
Update Alice's age to 23

- DELETE - Removes records from a table.

```
DELETE FROM students
WHERE id = 1;
```
Delete record with id = 1

- SELECT - Retrieves data from a table.

```
SELECT * FROM students;
```
Get all records

8. Filtering Data - WHERE Clause

The WHERE clause is used to filter records based on a condition.

```
SELECT * FROM students WHERE age > 18;                  # Age greater than 18
SELECT name, age FROM students WHERE age <= 21;         # Age less than or equal to 21
SELECT * FROM students WHERE name = 'Alice';            # Name is Alice
SELECT * FROM students WHERE age BETWEEN 18 AND 25;     # Age between 18 and 25
SELECT * FROM students WHERE name LIKE 'A%';            # Name starts with A
```

9. Sorting Data - ORDER BY Clause

ORDER BY clause is used to sort the result set.

```
SELECT * FROM students ORDER BY name ASC;               # Sort name A to Z
SELECT * FROM students ORDER BY age DESC;               # Sort age Z to A
SELECT name, age FROM students ORDER BY age ASC, name DESC;  # Sort by age, then name (desc)
```

10. GROUP BY Clause

GROUP BY clause is used to group rows that have the same values in specified columns. It is often used with aggregate functions.

Example:

```
SELECT department, COUNT(*) AS total_employees
FROM employees
GROUP BY department;
```

Count total employees in each department

11. HAVING Clause

HAVING clause is used to filter groups based on a condition. It works like WHERE but with GROUP BY.

Example:

```
SELECT department, COUNT(*) AS total_employees
FROM employees
GROUP BY department
HAVING COUNT(*) > 5;
```

Show departments having more than 5 employees

12. ORDER BY Clause

ORDER BY clause is used to sort the result set either in ascending (ASC) or descending (DESC) order.

Example:

```
SELECT name, salary
FROM employees
ORDER BY salary DESC;
```

Sort employees by salary from high to low

13. TOP, LIMIT and OFFSET

- TOP (SQL Server): Used to limit the number of rows returned.
- LIMIT (MySQL, PostgreSQL, SQLite): Limits the number of rows.
- OFFSET : Skips a specified number of rows before starting to return results.

Examples:

Using TOP (SQL Server)

```
SELECT TOP 5 name, salary
FROM employees
ORDER BY salary DESC;
```

Using LIMIT and OFFSET (MySQL)

```
SELECT name, salary
FROM employees
ORDER BY salary DESC
LIMIT 5 OFFSET 10;
```

Skip first 10 rows and return next 5 rows

14. DISTINCT Keyword

DISTINCT keyword is used to return only unique (different) values.

Example:

```
SELECT DISTINCT department
FROM employees;
```

Returns unique departments

15. Aliases

Aliases are used to give a table or column a temporary name.

Example:

```
SELECT e.name AS employee_name, e.salary AS emp_salary
FROM employees AS e;
```

e is alias for table employees

16. NULL Values

NULL represents a missing, unknown, or not applicable value.

- IS NULL : Finds records with NULL values.
- IS NOT NULL : Finds records with non-NULL values.

Examples:

```
SELECT *
FROM employees
WHERE email IS NULL;
```

Emails that are missing

```
SELECT *
FROM employees
WHERE email IS NOT NULL;
```

Emails that are available

17. Wildcards

Wildcards are used with LIKE operator to search for a pattern in a column.

- % : Represents zero, one, or multiple characters.
- _ : Represents a single character.

Examples:

```
SELECT *
FROM employees
WHERE name LIKE 'A%';
```

Names starting with A

```
SELECT *
FROM employees
WHERE name LIKE '_o%';
```

Names with second letter o

18. IN Operator

IN operator is used to check a value against a list of possible values.

Example:

```
SELECT name, department
FROM employees
WHERE department IN ('HR', 'IT', 'Finance');
```

Employees from HR, IT or Finance departments

19. BETWEEN Operator

BETWEEN operator is used to filter the result set within a given range.
The range values can be numbers, text, or dates.

Example:

```
SELECT name, salary
FROM employees
WHERE salary BETWEEN 30000 AND 60000;
```

Returns employees whose salary is between 30000 and 60000

20. LIKE Operator

LIKE operator is used in a WHERE clause to search for a specified pattern in a column.

- % : Represents zero, one, or multiple characters.
- _ : Represents a single character.

Examples:

```
SELECT name
FROM employees
WHERE name LIKE 'A%';
# Names starting with A
```

Matches: Alice , Aaron, Andrew, etc.

```
SELECT name
FROM employees
WHERE name LIKE '_o%';
# Names with second letter as 'o'
```

Matches: Bob, Tom, Robert, etc.

21. IN Operator

IN operator is used to check a value against a list of values.

Example:

```
SELECT name, department
FROM employees
WHERE department IN ('HR', 'IT', 'Finance');
```

Employees working in HR, IT or Finance departments

22. IS NULL and IS NOT NULL

These operators are used to check for NULL values in a column.

Examples:

```
SELECT name, email
FROM employees
WHERE email IS NULL;
# Returns employees whose email is missing
```

```
SELECT name, email
FROM employees
WHERE email IS NOT NULL;
# Returns employees whose email is available
```

23. Aggregate Functions

Aggregate functions perform a calculation on a set of values and return a single value.
They are often used with GROUP BY clause.

Common Aggregate Functions:

- COUNT() : Returns the number of rows.
- SUM() : Returns the total sum of a numeric column.
- AVG() : Returns the average of a numeric column.
- MIN() : Returns the minimum value.
- MAX() : Returns the maximum value.

Example:

```
SELECT department, AVG(salary) AS avg_salary
FROM employees
GROUP BY department;
# Average salary in each department
```

- Subqueries can be used in SELECT, FROM, WHERE, and HAVING clauses.
- They can return a single value, multiple values, or a table.
- Types : Single-row subquery
  Multiple-row subquery
  Correlated subquery

Examples:

1) Single-row subquery

```
SELECT name
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

Employees earning more than the average salary

2) Multiple-row subquery

```
SELECT name
FROM employees
WHERE department_id IN (SELECT id FROM departments);
```

Employees working in existing departments

3) Correlated subquery

```
SELECT e.name
FROM employees e
WHERE salary > (SELECT AVG(salary)
                FROM employees
                WHERE department_id = e.department_id);
```

Compares salary with average salary in the same department

25. Unions

UNION operator is used to combine the result sets of two or more SELECT statements. The columns in each SELECT must be in the same order and have compatible data types.

Example:

```
SELECT name, email FROM employees
UNION
SELECT name, email FROM contractors;
```

Combines employees and contractors (duplicates are removed)

26. UNION ALL

UNION ALL is similar to UNION, but it includes duplicate rows in the result set.

Example:

```
SELECT name, email FROM employees
UNION ALL
SELECT name, email FROM contractors;
```

Combines all rows including duplicates

Note: Use UNION when you want unique results.

27. INTERSECT

INTERSECT operator returns the common rows from two SELECT statements.

Example:

```
SELECT name, email FROM employees
INTERSECT
SELECT name, email FROM contractors;
```

Returns only the rows that are common in both result sets

28. EXCEPT (or MINUS in some databases)

EXCEPT operator returns the rows from the first SELECT statement that are not present in the second SELECT statement.

Example:

```
SELECT name, email FROM employees
EXCEPT
SELECT name, email FROM contractors;
```

Returns the rows that are in employees but not in contractors

29. CROSS JOIN

CROSS JOIN returns the Cartesian product of both tables (each row from first table is combined with each row from second table).

Example:

```
SELECT e.name, d.department_name
FROM employees e
CROSS JOIN departments d;
```

Returns all possible combinations of employees and departments

30. SELF JOIN

SELF JOIN is a regular join, but the table is joined with itself.

Example:

```
SELECT e1.name AS employee, e2.name AS manager
FROM employees e1
JOIN employees e2
ON e1.manager_id = e2.employee_id;
```

Shows employee and their manager from the same table

Note: Set operators (UNION, INTERSECT, EXCEPT) require both SELECT statements to have the same number of columns with compatible data types.

31. NATURAL JOIN

NATURAL JOIN automatically joins tables based on columns that have the same name in both tables.

Example:

```
SELECT  *
FROM    employees
NATURAL JOIN  departments;
```

Joins tables on columns with the same name

32. USING Clause

USING clause is used in JOIN to specify the column(s) to join on.

Example:

```
SELECT  e.name, d.department_name
FROM    employees e
JOIN    departments d
USING   (department_id);
```

Joins using the common column department_id

33. EXISTS Operator

EXISTS operator tests for the existence of any record in a subquery.

Example:

```
SELECT  name
FROM    employees e
WHERE   EXISTS  (SELECT 1 FROM departments d
                 WHERE d.department_id = e.department_id);
```

Returns employees who have a matching department

34. NOT EXISTS Operator

NOT EXISTS returns rows from the outer query for which the subquery returns no rows.

Example:

```
SELECT  name
FROM    employees e
WHERE   NOT EXISTS (SELECT 1 FROM departments d
                    WHERE d.department_id = e.department_id);
```

Returns employees who are not assigned to any department

35. IN Operator

IN operator allows you to specify multiple values in a WHERE clause.

Example:

```
SELECT name, department
FROM employees
WHERE department IN ('HR', 'IT', 'Finance');
```

Returns employees from specified departments

36. NOT IN Operator

NOT IN operator is used to exclude rows that match any of the values in a list.

Example:

```
SELECT name, department
FROM employees
WHERE department NOT IN ('HR', 'IT');
```

Returns employees not in HR or IT departments

37. ANY and ALL Operators

ANY and ALL are used with subqueries to compare a value with any or all values returned by the subquery.

- ANY : Returns true if the condition is true for any value.

  Example:

```
SELECT name, salary
FROM employees
WHERE salary > ANY (SELECT salary FROM employees WHERE department = 'IT');
```

Salary greater than any salary in IT department

- ALL : Returns true if the condition is true for all values.

  Example:

```
SELECT name, salary
FROM employees
WHERE salary > ALL (SELECT salary FROM employees WHERE department = 'IT');
```

Salary greater than all salaries in IT department

38. SOME Operator

SOME is another name for ANY. It works exactly the same.

Example:

```
SELECT name, salary
FROM employees
WHERE salary > SOME (SELECT salary FROM employees WHERE department = 'IT');
```

Same as using ANY operator

Note: ANY, SOME, and ALL are used with subqueries and are not commonly used in everyday SQL queries.

39. GROUP BY Clause

GROUP BY clause is used with aggregate functions to group the result-set by one or more columns.

Example:

```
SELECT department_id, COUNT(*) AS total_employees
FROM employees
GROUP BY department_id;
```

Groups employees by department and counts total employees

40. HAVING Clause

HAVING clause is used to filter groups created by GROUP BY.

Example:

```
SELECT department_id, COUNT(*) AS total_employees
FROM employees
GROUP BY department_id
HAVING COUNT(*) > 5;
```

Displays departments having more than 5 employees

Note: WHERE filters rows before grouping.
HAVING filters groups after grouping.

41. ORDER BY Clause

ORDER BY clause is used to sort the result-set in ascending (ASC) or descending (DESC) order.

Example:

```
SELECT name, salary
FROM employees
ORDER BY salary DESC;
```

Sorts employees by salary in descending order

42. TOP Clause

TOP clause is used to specify the number of rows to return from the result-set.

Example:

```
SELECT TOP 5 name, salary
FROM employees
ORDER BY salary DESC;
```

Returns top 5 highest salaries

Note: TOP is specific to SQL Server (not standard SQL).

43. DISTINCT Keyword

DISTINCT keyword is used to return only distinct (different) values.

Example:

```
SELECT DISTINCT department
FROM employees;
```

Returns unique departments without duplicates

44. LIMIT / OFFSET Clause

LIMIT is used to restrict the number of rows returned.

OFFSET is used to skip a number of rows before starting to return rows.

Example:

```
SELECT name, salary
FROM employees
ORDER BY salary DESC
LIMIT 10 OFFSET 20;
```

Skips first 20 rows and returns next 10 rows

45. UNION Operator

UNION operator is used to combine the result-set of two or more SELECT statements.

It returns only distinct rows by default.

Example:

```
SELECT name FROM employees
UNION
SELECT name FROM contractors;
```

Combines names from employees and contractors (removes duplicates)

Note: Use UNION ALL if you want to include duplicate rows.

46. CASE Expression

CASE expression is used to return different values based on conditions.

Example:

```
SELECT name, salary,
CASE
    WHEN salary >= 50000 THEN 'High'
    WHEN salary BETWEEN 30000 AND 50000 THEN 'Medium'
    ELSE 'Low'
END AS salary_level
FROM employees;
```

Categorizes salary into High, Medium or Low

47. BETWEEN Operator

BETWEEN operator is used to select values within a given range.

Example:

```
SELECT name, salary
FROM employees
WHERE salary BETWEEN 30000 AND 50000;
```

Returns employees whose salary is between 30000 and 50000

48. LIKE Operator

LIKE operator is used in a WHERE clause to search for a specified pattern in a column.

Example:

```
SELECT name
FROM employees
WHERE name LIKE 'A%';
```

Returns names that start with 'A'

- % → Represents zero, one or more characters
- _ → Represents a single character

49. IN Operator

IN operator allows you to specify multiple values in a WHERE clause.

Example:

```
SELECT name, department
FROM employees
WHERE department IN ('HR', 'IT', 'Finance');
```

Returns employees from specified departments

50. IS NULL / IS NOT NULL Operator

IS NULL operator is used to check for NULL values.
IS NOT NULL operator is used to check for non-NULL values.

Example:

```
-- IS NULL
SELECT name, email
FROM employees
WHERE email IS NULL;
```

Returns employees whose email is NULL

```
-- IS NOT NULL
SELECT name, email
FROM employees
WHERE email IS NOT NULL;
```

Returns employees whose email is not NULL

51. EXISTS Operator

EXISTS operator is used to test for the existence of any record in a subquery.

Example:

```
SELECT name
FROM employees e
WHERE EXISTS (SELECT 1
              FROM departments d
              WHERE d.id = e.department_id );
```

Returns employees for whom a matching department exists

52. ANY / SOME Operator

ANY / SOME operator is used with a subquery to compare a value with any value returned by the subquery.

ANY and SOME are synonyms.

Example:

```
SELECT name, salary
FROM employees
WHERE salary > ANY (SELECT salary FROM employees
                    WHERE department_id = 10);
```

Returns employees whose salary is greater than any salary in department 10

53. ALL Operator

ALL operator is used to compare a value with all values returned by the subquery.

Example:

```
SELECT name, salary
FROM employees
WHERE salary > ALL (SELECT salary FROM employees
                    WHERE department_id = 10);
```

Returns employees whose salary is greater than all salaries in department 10

54. COALESCE() Function

COALESCE() returns the first non-NULL value from a list of expressions.

Example:

```
SELECT name,
    COALESCE(phone, mobile, 'N/A') AS contact_no
FROM employees;
```

Returns phone if available, else mobile, else 'N/A'

Note: COALESCE() is very useful when dealing with NULL values in multiple columns.